

Katalon Semi-Auto Capture Template

Template copy-paste untuk membuat DevLog lebih semi otomatis: user setup sekali di project Katalon, lalu testcase cukup run dengan mapping UUID QA Desk.

TARGET

QA bisa run testcase Katalon tanpa copy setup DevLog di setiap script.

YANG OTOMATIS

Start, finish, fail, relay send, mapping UUID, browser log fallback.

YANG TETAP MANUAL

Step penting tetap dipanggil dengan DevLog.step(), API JVM bisa memakai DevLog.network().

Struktur file yang disarankan

```
Keywords/com/Utils/DevLog.groovy
Test Listeners/DevLogListener.groovy
Include/testcase-map.json
Profiles/default.glbl: webQaRelayUrl, webQaEnabled
```

OVERVIEW

Alur semi otomatis

1. Copy **DevLog.groovy** ke folder **Keywords/com/webqa**.
2. Copy **DevLogListener.groovy** ke **Test Listeners**.
3. Buat mapping **Include/testcase-map.json**.
4. Tambahkan GlobalVariable **webQaRelayUrl** dan **webQaEnabled**.
5. Di testcase, panggil **DevLog.attachBrowser()** setelah browser dibuka.
6. Untuk step penting, panggil **DevLog.step("...")**.
7. Untuk API call dari JVM/HttpHelper, panggil **DevLog.network(...)** atau modifikasi HttpHelper agar otomatis.

Global Variable yang dibutuhkan

NAMA	CONTOH NILAI	FUNGSI
webQaRelayUrl	http://127.0.0.1:3001/log	Endpoint relay DevLog.
webQaEnabled	true	Bisa mematikan DevLog tanpa edit script.

DEVLOG.GROOVY

Keywords/com/Utils/DevLog.groovy

```
package com.utils

import com.kms.katalon.core.annotation.Keyword
import com.kms.katalon.core.configuration.RunConfiguration
import com.kms.katalon.core.webui.driver.DriverFactory
import com.kms.katalon.core.webui.keyword.WebUiBuiltInKeywords as WebUI
import groovy.json.JsonOutput
import groovy.json.JsonSlurper
import internal.GlobalVariable
import org.apache.http.client.config.RequestConfig
import org.apache.http.client.methods.HttpPost
import org.apache.http.entity.StringEntity
import org.apache.http.impl.client.CloseableHttpClient
import org.apache.http.impl.client.HttpClients
import org.openqa.selenium.logging.LogEntry
import org.openqa.selenium.logging.LogType
import org.openqa.selenium.devtools.DevTools
import org.openqa.selenium.devtools.HasDevTools
import java.text.SimpleDateFormat
import java.util.Optional
import java.util.concurrent.ConcurrentLinkedQueue
import java.util.function.Consumer

class DevLog {
    private static String relayUrl = ''
```

```

private static String testCaseId = ''
private static String katalonPath = ''
private static long startedAtMs = 0L
private static DevTools devTools = null
private static ConcurrentLinkedQueue cdpEventQueue = new ConcurrentLinkedQueue()
private static volatile boolean cdpWorkerRunning = false

@Keyword
def start(def context) {
    if (!isEnabled()) return
    relayUrl = getRelayUrl()
    katalonPath = context?.testCaseId ?: context?.getTestCaseId() ?: ''
    testCaseId = resolveMappedTestCaseId(katalonPath)
    startedAtMs = System.currentTimeMillis()
    sendStructuredLog('execution', [
        message: "Starting Automation: ${katalonPath ?: 'Unknown Katalon Test Case'}"
    ])
}

@Keyword
def finish(def context) {
    if (!isEnabled()) return
    drainCdpQueue()
    sniffBrowserLogs()
    String status = context?.testCaseStatus ?: context?.getTestCaseStatus() ?: 'UNKNOWN'
    long durationMs = startedAtMs ? System.currentTimeMillis() - startedAtMs : 0L
    sendStructuredLog('execution', [
        message: "Execution complete. Status: ${status}. Duration: ${durationMs}ms"
    ])
    devTools = null
    cdpWorkerRunning = false
}

@Keyword
def step(String message) {
    writeLocalComment(message)
    sendStructuredLog('execution', [message: message])
    maybeSniffLogs()
}

@Keyword
def console(String level, String message) {
    sendStructuredLog('console', [level: level ?: 'INFO', message: message ?: ''])
}

@Keyword
def network(String method, String url, def responseOrData = null) {
    try {
        Map data = [
            event: 'Response',
            method: (method ?: 'TRACE').toString().toUpperCase(),
            url: url ?: '',
            status: extractStatus(responseOrData),
            success: extractSuccess(responseOrData),
            data: extractBody(responseOrData)
        ]
        sendStructuredLog('network', data)
    } catch (Exception e) {
        System.out.println('[DevLog.network] ' + e.message)
    }
}

@Keyword
def attachBrowser() {
    if (!isEnabled()) return
    initDevTools()
}

@Keyword
def setTestCaseId(String id) {
    testCaseId = id ?: testCaseId
    System.setProperty('webqa.testCaseId', testCaseId)
}

static boolean isEnabled() {
    try {
        return String.valueOf(GlobalVariable.webQaEnabled ?: 'true').toBoolean()
    } catch (Exception ignored) {
        return true
    }
}

static String getRelayUrl() {
    try {
        return String.valueOf(GlobalVariable.webQaRelayUrl ?: 'http://127.0.0.1:3001/log')
    } catch (Exception ignored) {
        return 'http://127.0.0.1:3001/log'
    }
}

static String resolveMappedTestCaseId(String path) {
    String override = System.getProperty('webqa.testCaseId')
    if (override) return override

    File mapFile = new File(RunConfiguration.getProjectDir(), 'Include/testcase-map.json')
    if (!mapFile.exists()) {
        System.out.println('[DevLog] Mapping file not found: ' + mapFile.absolutePath)
        return path
    }

    try {
        def map = new JsonSlurper().parse(mapFile)
        String mapped = map[path] ?: map[path?.replace('Test Cases/', '')]
        if (mapped) {
            System.setProperty('webqa.testCaseId', mapped)
            return mapped
        }
        System.out.println('[DevLog] Mapping not found for: ' + path)
    } catch (Exception e) {
        System.out.println('[DevLog] Failed reading mapping: ' + e.message)
    }
    return path
}

```

```

static void sendStructuredLog(String category, Map data) {
    if (!isEnabled()) return
    try {
        String id = testCaseId ?: System.getProperty('webqa.testCaseId') ?: katalonPath
        if (id) {
            System.out.println('[DevLog] testCaseId is empty. Log skipped.')
            return
        }

        Map payload
        switch (category) {
            case 'console':
                payload = [
                    type: 'log',
                    testCaseId: id,
                    level: data.level ?: 'INFO',
                    log: data.message ?: '',
                    relativeMs: relativeMs()
                ]
                break
            case 'network':
                payload = [
                    type: 'log',
                    testCaseId: id,
                    network: data,
                    log: 'Network Trace',
                    relativeMs: relativeMs()
                ]
                break
            case 'execution':
            default:
                payload = [
                    type: 'log',
                    testCaseId: id,
                    log: data.message ?: '',
                    relativeMs: relativeMs()
                ]
                break
        }

        RequestConfig config = RequestConfig.custom()
            .setConnectTimeout(1000)
            .setSocketTimeout(1000)
            .setConnectionRequestTimeout(1000)
            .build()

        HttpPost post = new HttpPost(relayUrl ?: getRelayUrl())
        post.setHeader('Content-Type', 'application/json')
        post.setEntity(new StringEntity(JsonOutput.toJson(payload), 'UTF-8'))
        CloseableHttpClient client = HttpClients.custom()
            .setDefaultRequestConfig(config)
            .build()
        client.execute(post).close()
        client.close()
    } catch (Exception e) {
        System.out.println('[DevLog] Relay offline or send failed: ' + e.message)
    }
}

static void writeLocalComment(String message) {
    String timestamp = new SimpleDateFormat('yyyy-MM-dd HH:mm:ss.SSS').format(new Date())
    String formatted = "[${timestamp}] ${message}"
    try { WebUI.comment(message) } catch (Exception ignored) {}
    System.out.println(formatted)
}

static long relativeMs() {
    return startedAtMs ? Math.max(0L, System.currentTimeMillis() - startedAtMs) : 0L
}

static Integer extractStatus(def responseOrData) {
    try {
        if (responseOrData instanceof Map && responseOrData.status != null) return responseOrData.status as Integer
        if (responseOrData?.statusCode != null) return responseOrData.statusCode as Integer
    } catch (Exception ignored) {}
    return 0
}

static Boolean extractSuccess(def responseOrData) {
    try {
        if (responseOrData instanceof Map && responseOrData.success != null) return responseOrData.success as Boolean
        if (responseOrData?.success != null) return responseOrData.success as Boolean
        Integer status = extractStatus(responseOrData)
        return status >= 200 && status < 400
    } catch (Exception ignored) {}
    return false
}

static Object extractBody(def responseOrData) {
    try {
        def body = responseOrData?.responseText ?: responseOrData?.data ?: responseOrData
        String text = body == null ? '' : String.valueOf(body)
        return text.length() > 3000 ? text.substring(0, 3000) + '... [truncated]': text
    } catch (Exception ignored) {
        return ''
    }
}

static void maybeSniffLogs() {
    if (devTools == null) sniffBrowserLogs()
}

static void sniffBrowserLogs() {
    try {
        def driver = DriverFactory.getWebDriver()
        def entries = driver.manage().logs().get(LogType.BROWSER).all
        entries?.each { LogEntry entry ->
            sendStructuredLog('console', [
                level: entry.level?.name()?.toUpperCase() ?: 'INFO',
                message: '[BrowserLog Fallback] ' + entry.message
            ])
        }
    } catch (Exception ignored) {}
}

```

```

static void initDevTools() {
    try {
        def driver = DriverFactory.getWebDriver()
        if (!(driver instanceof HasDevTools)) {
            sendStructuredLog('console', [level: 'WARNING', message: 'Browser driver does not support DevTools.'])
            return
        }

        devTools = ((HasDevTools) driver).getDevTools()
        devTools.createSession()

        String version = ['v143', 'v138', 'v129', 'v128', 'v120', 'v112', 'v85'].find { v ->
            try { Class.forName("org.openqa.selenium.devtools.${v}.network.Network"); return true }
            catch (Exception ignored) { return false }
        }
        if (!version) {
            sendStructuredLog('console', [level: 'WARNING', message: 'No selenium-devtools version found. Fallback browser logs only.'])
            devTools = null
            return
        }

        def netClass = Class.forName("org.openqa.selenium.devtools.${version}.network.Network")
        def conClass = Class.forName("org.openqa.selenium.devtools.${version}.console.Console")

        try {
            def netEnable = netClass.methods.find { it.name == 'enable' }
            Object[] netArgs = new Object[netEnable.parameterTypes.length]
            for (int i = 0; i < netArgs.length; i++) {
                netArgs[i] = netEnable.parameterTypes[i] == Optional ? Optional.empty() : null
            }
            devTools.send(netEnable.invoke(null, netArgs))
            def conEnable = conClass.methods.find { it.name == 'enable' }
            if (conEnable) devTools.send(conEnable.invoke(null))
        } catch (Exception ignored) {}

        registerListener(netClass, 'requestWillBeSent') { ev ->
            try {
                def req = ev.request
                queueCdp('network', [
                    event: 'Request',
                    method: req?.method,
                    url: req?.url,
                    headers: req?.headers?.toString()?.take(800)
                ])
            } catch (Exception ignored) {}
        }

        registerListener(netClass, 'responseReceived') { ev ->
            try {
                def res = ev.response
                queueCdp('network', [
                    event: 'Response',
                    method: ev.type?.toString(),
                    url: res?.url,
                    status: res?.status as Integer,
                    success: (res?.status as Integer) < 400,
                    headers: res?.headers?.toString()?.take(800)
                ])
            } catch (Exception ignored) {}
        }

        registerListener(conClass, 'messageAdded') { ev ->
            try {
                def msg = ev.message
                queueCdp('console', [
                    level: msg?.level?.toString()?.toUpperCase() ?: 'INFO',
                    message: msg?.text?.toString() ?: String.valueOf(msg)
                ])
            } catch (Exception ignored) {}
        }

        startCdpWorker()
        sendStructuredLog('console', [level: 'INFO', message: "DevTools attached using ${version}."])
    } catch (Exception e) {
        devTools = null
        sendStructuredLog('console', [level: 'WARNING', message: 'DevTools init failed: ' + e.message])
    }
}

static void registerListener(def domainClass, String eventMethod, Closure callback) {
    try {
        def eventObj = domainClass.methods.find { it.name == eventMethod && it.parameterTypes.length == 0 }.invoke(null)
        if (!eventObj) return
        def addMethod = devTools.class.methods.find {
            it.name == 'addListener' && it.parameterTypes.length == 2 && it.parameterTypes[1].name.contains('Consumer')
        }
        if (!addMethod) return
        addMethod.invoke(devTools, eventObj, new Consumer<Object>() {
            @Override void accept(Object ev) { callback(ev) }
        })
    } catch (Exception e) {
        System.out.println("[DevLog] Listener failed for " + eventMethod + ": " + e.message)
    }
}

static void queueCdp(String category, Map data) {
    cdpEventQueue.offer([category: category, data: data])
}

static void startCdpWorker() {
    if (cdpWorkerRunning) return
    cdpWorkerRunning = true
    Thread.startDaemon('DevLog-CDP-Worker') {
        while (cdpWorkerRunning) {
            drainCdpQueue()
            Thread.sleep(250)
        }
    }
}

static void drainCdpQueue() {
    while (!cdpEventQueue.isEmpty()) {
        def event = cdpEventQueue.poll()
    }
}

```

```

    }
    if (event) sendStructuredLog(event.category as String, event.data as Map)
  }
}

```

TEST LISTENER

Test Listeners/DevLogListener.groovy

```

import com.kms.katalon.core.annotation.BeforeTestCase
import com.kms.katalon.core.annotation.AfterTestCase
import com.kms.katalon.core.context.TestCaseContext

class DevLogListener {
    @BeforeTestCase
    def beforeTestCase(TestCaseContext testCaseContext) {
        CustomKeywords.'com.utils.DevLog.start'(testCaseContext)
    }

    @AfterTestCase
    def afterTestCase(TestCaseContext testCaseContext) {
        CustomKeywords.'com.utils.DevLog.finish'(testCaseContext)
    }
}

```

MAPPING JSON

Include/testcase-map.json

```

{
  "Test Cases/M-Servios/POS Order Flow": "00000000-0000-0000-0000-000000000001",
  "Test Cases/M-Servios/Login": "GANTI_DENGAN_UUID_QA_DESK",
  "Test Cases/Regression/Checkout": "GANTI_DENGAN_UUID_QA_DESK"
}

```

Kiri adalah path testcase di Katalon. Kanan adalah Internal Database ID / UUID dari QA Desk, bukan display ID seperti E-124.

CONTOH TESTCASE

Contoh script testcase setelah semi otomatis

```

import static com.kms.katalon.core.testobject.ObjectRepository.findTestObject
import com.kms.katalon.core.webui.keyword.WebUiBuiltInKeywords as WebUI
import internal.GlobalVariable as GlobalVariable
import groovy.json.JsonSlurper

String apiBase = GlobalVariable.baseUrl
String posLogin = 'sammy'
String posPin = '1234'

CustomKeywords.'com.utils.DevLog.step'('Open browser and attach DevLog capture')
WebUI.openBrowser('')
WebUI.maximizeWindow()
CustomKeywords.'com.utils.DevLog.attachBrowser'()

CustomKeywords.'com.utils.DevLog.step'('Login POS')
String loginUrl = "${apiBase}/api/pos/login"
def loginResp = CustomKeywords.'com.utils.HttpHelper.postJson'(loginUrl, [login: posLogin, pin: posPin])
CustomKeywords.'com.utils.DevLog.network'('POST', loginUrl, loginResp)

if (!loginResp.success) {
    throw new Exception("POS Login failed. Status: ${loginResp.statusCode}")
}

def loginData = new JsonSlurper().parseText(loginResp.responseText).data
String companyToken = loginData.access_token

CustomKeywords.'com.utils.DevLog.step'('Navigate to target app')
WebUI.navigateToUrl(GlobalVariable.targetUrl)
WebUI.waitForPageLoad(10)

CustomKeywords.'com.utils.DevLog.step'('Click submit order')
WebUI.click(findTestObject('Object Repository/Checkout/Button_SubmitOrder'))

CustomKeywords.'com.utils.DevLog.step'('Validate order success message')
WebUI.verifyElementPresent(findTestObject('Object Repository/Checkout/Success_Message'), 10)

WebUI.closeBrowser()

```

HTTPHELPER OPSIONAL

Optional: auto network log di HttpHelper

Jika semua API call memakai HttpHelper, tambahkan panggilan DevLog di dalam HttpHelper supaya testcase tidak perlu memanggil DevLog.network() berkali-kali.

```
// Contoh pola yang ditambahkan setelah response didapat di HttpHelper.  
// Sesuaikan nama variable response dengan implementasi HttpHelper kamu.  
  
try {  
    CustomKeywords.'com.utils.DevLog.network'(method, url, response)  
} catch (Exception ignored) {  
    // DevLog tidak boleh membuat API helper gagal.  
}
```

Catatan: template ini sengaja semi otomatis. Start/finish/fail dibuat otomatis oleh listener, tetapi step bisnis tetap sebaiknya ditulis eksplisit agar DevLog mudah dibaca manusia.